

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	N.KISHAN KUMAR	Reg. No.:	192424404
Section / Batch:	B.tech AI & DS	Date:	08-08-2026

Code of Conduct

I N.Kishan kumar (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: SMART MOBILE KEYBOARD PREDICTION SYSTEM

Introduction

An AI-powered mobile keyboard predicts the next word based on previously typed words. N-gram language models such as Bigram and Trigram models estimate the probability of upcoming words using word sequences observed in a training corpus. Backoff and interpolation techniques help the system handle unseen or rare sequences, while entropy measures the uncertainty associated with predictions.

Given Corpus:

“Data science is powerful, data science drives innovation, data science is evolving.”

Given Statistics:

$C(\text{data}) = 3$

$C(\text{data science}) = 3$

$C(\text{science is}) = 2$

$C(\text{science drives}) = 1$

$C(\text{science drives innovation}) = 1$

Interpolation Weights:

$\lambda_1 = 0.5$ – Trigram

$\lambda_2 = 0.3$ – Bigram

$\lambda_3 = 0.2$ – Unigram

Prediction Probabilities:

$P(\text{is}) = 0.66$

$P(\text{drives}) = 0.33$

1. Maximum Likelihood Estimate of Bigram Probability

The Maximum Likelihood Estimate of a bigram is calculated using:

$P(w_2 | w_1) = C(w_1, w_2) / C(w_1)$

For the bigram “data science”:

$P(\text{science} | \text{data}) = C(\text{data science}) / C(\text{data})$

Given:

$C(\text{data science}) = 3$

$C(\text{data}) = 3$

Therefore,

$P(\text{science} | \text{data}) = 3 / 3$

$P(\text{science} | \text{data}) = 1.0$

Interpretation

The probability of 1.0 means that in the given training corpus, every occurrence of the word “data” is followed by the word “science”.

Therefore, when a user types:

data → ?

the mobile keyboard will strongly predict:

data → **science**

This demonstrates how bigram probabilities allow predictive keyboards to suggest the most likely next word based on the immediately preceding word. However, the probability is based only on the available training corpus, so a larger and more diverse corpus is required for realistic applications.

2. Backoff Model for an Unseen Sequence

Consider the unseen sequence:

“data science improves”

The trigram probability is:

$P(\text{improves} \mid \text{data science})$

The trigram “data science improves” does not occur in the given corpus.

Therefore,

$C(\text{data science improves}) = 0$

and direct trigram estimation cannot provide a useful probability.

Applying Backoff

The model first attempts the trigram:

$P(\text{improves} \mid \text{data science})$

Since it is unseen, the model backs off to the bigram:

$P(\text{improves} \mid \text{science})$

The bigram “science improves” is also not present in the corpus.

Therefore, the model backs off again to the unigram:

$P(\text{improves})$

If “improves” is also absent from the training vocabulary, its unsmoothed unigram probability is:

$P(\text{improves}) = 0$

Hence, using the supplied corpus alone:

$P(\text{data science improves}) = 0$

In a practical language model, smoothing or an <UNK> unknown-word probability would normally be used so that the sequence receives a small non-zero probability.

Importance of Backoff

Backoff is essential because users frequently type word combinations that are not present in the training dataset. It allows the system to move from a specific higher-order model to a more general lower-order model.

For example:

Trigram $\rightarrow$ Bigram $\rightarrow$ Unigram

This improves coverage, prevents the prediction mechanism from failing on unseen sequences, and supports real-time prediction even when the available language data is sparse.

3. Deleted Interpolation for “data science is”

Deleted interpolation combines Trigram, Bigram, and Unigram probabilities.

The general formula is:

$$P(w_3 \mid w_1, w_2) = \lambda_1 P(w_3 \mid w_1, w_2) + \lambda_2 P(w_3 \mid w_2) + \lambda_3 P(w_3)$$

Given:

$$\lambda_1 = 0.5$$

$$\lambda_2 = 0.3$$

$$\lambda_3 = 0.2$$

For the sequence:

data science is

Step 1: Trigram Probability

The corpus contains:

“data science is” = 2 times

and:

“data science” = 3 times

Therefore,

$$P(\text{is} \mid \text{data science}) = 2 / 3$$

$$P(\text{is} \mid \text{data science}) = 0.667$$

Step 2: Bigram Probability

$$C(\text{science is}) = 2$$

The word “science” occurs 3 times.

$$P(\text{is} \mid \text{science}) = 2 / 3$$

$$P(\text{is} \mid \text{science}) = 0.667$$

Step 3: Unigram Probability

Ignoring punctuation, the corpus contains 12 word tokens:

Data science is powerful

data science drives innovation

data science is evolving

The word “is” occurs 2 times.

$$P(\text{is}) = 2 / 12$$

$$P(\text{is}) = 0.167$$

Step 4: Apply Interpolation

$$P(\text{is} \mid \text{data science})$$

$$= (0.5 \times 0.667) + (0.3 \times 0.667) + (0.2 \times 0.167)$$

$$= 0.3335 + 0.2001 + 0.0334$$

$$P(\text{is} \mid \text{data science}) \approx 0.567$$

Interpretation

The interpolated probability is approximately:

$$0.567 \text{ or } 56.7\%$$

Interpolation is useful because it does not depend completely on a single N-gram level. It combines information from multiple models.

The Trigram model provides more contextual information, the Bigram model provides better coverage when trigram evidence is limited, and the Unigram model provides general word-frequency information. Therefore, interpolation achieves a balance between **reliability and coverage**, especially when the training dataset is sparse.

4. Entropy Analysis

Given:

$$P(\text{is}) = 0.66$$

$$P(\text{drives}) = 0.33$$

Entropy is calculated using:

$$H(X) = -\sum P(x) \log_2 P(x)$$

Therefore,

$$H = -[0.66 \log_2(0.66) + 0.33 \log_2(0.33)]$$

Approximately,

$$\log_2(0.66) \approx -0.599$$

$$\log_2(0.33) \approx -1.599$$

Therefore,

$$H \approx -[(0.66 \times -0.599) + (0.33 \times -1.599)]$$

$$H \approx 0.395 + 0.528$$

$$H \approx 0.923 \text{ bits}$$

The supplied probabilities total 0.99 because they are rounded values, so this is an approximate entropy.

Interpretation

An entropy of approximately **0.92 bits** indicates moderate uncertainty in the next-word prediction.

The model prefers:

science → **is (0.66)**

over:

science → **drives (0.33)**

However, the probability of “drives” is still significant. Therefore, the model is not completely certain about its prediction.

Lower entropy indicates higher prediction confidence, whereas higher entropy indicates greater uncertainty.

In intelligent keyboards, entropy can help determine whether the system should confidently display one prediction or provide multiple possible word suggestions.

Conclusion

The mobile keyboard prediction system demonstrates how Bigram and Trigram models predict words from context. MLE calculates probabilities from observed frequencies, Backoff handles unseen sequences, Deleted Interpolation combines multiple N-gram levels, and Entropy measures prediction uncertainty. Together, these techniques improve the reliability and coverage of intelligent predictive text systems.

CASE STUDY 2: AI-POWERED CUSTOMER SUPPORT CHATBOT

Introduction

Part-of-Speech tagging assigns grammatical categories such as nouns, verbs, adjectives, and adverbs to words in a sentence. In a customer support chatbot, POS tagging helps the system understand sentence structure and resolve ambiguous words. Hidden Markov Models use transition and emission probabilities to select the most likely POS tag according to context.

1. Penn Treebank POS Tagging

Sentence 1

“Book a flight ticket now.”

POS Tags:

Book/VB

a/DT

flight/NN

ticket/NN

now/RB

Tagged Sentence:

Book/VB a/DT flight/NN ticket/NN now/RB

Explanation

“Book” is tagged as VB (Verb) because it appears at the beginning of an imperative sentence and represents the action of making a reservation.

Other tags are:

DT – Determiner

NN – Noun

RB – Adverb

Sentence 2

“This book is interesting.”

POS Tags:

This/DT
book/NN
is/VBZ
interesting/JJ
Tagged Sentence:
This/DT book/NN is/VBZ interesting/JJ

Explanation

In the second sentence, “book” represents an object or thing. Therefore, it functions as a Noun (NN).

Thus, the same word has different grammatical functions depending on its context:

Book a flight → Book = Verb (VB)

This book → book = Noun (NN)

This demonstrates lexical ambiguity in Natural Language Processing.

2. HMM Probability for “book” as a Verb

Given:

$$P(\text{book} \mid \text{VB}) = 0.6$$

$$P(\text{book} \mid \text{NN}) = 0.4$$

$$P(\text{Start} \rightarrow \text{VB}) = 0.5$$

$$P(\text{Start} \rightarrow \text{NN}) = 0.5$$

For Verb:

$$P(\text{VB}, \text{book}) = P(\text{Start} \rightarrow \text{VB}) \times P(\text{book} \mid \text{VB})$$

$$= 0.5 \times 0.6$$

$$= \mathbf{0.30}$$

For comparison, as a Noun:

$$P(\text{NN}, \text{book}) = P(\text{Start} \rightarrow \text{NN}) \times P(\text{book} \mid \text{NN})$$

$$= 0.5 \times 0.4$$

$$= \mathbf{0.20}$$

Therefore:

$$\text{Verb likelihood} = \mathbf{0.30}$$

$$\text{Noun likelihood} = \mathbf{0.20}$$

Since:

$$\mathbf{0.30 > 0.20}$$

the HMM favors the **VB (Verb)** tag.

Interpretation

The word “book” has a greater emission probability under the VB state than the NN state. Since both starting transition probabilities are equal, the VB path receives the higher likelihood.

The grammatical context also supports the result because “Book a flight ticket now” is an imperative command where “book” expresses an action.

3. Rule-Based Tagging vs Stochastic HMM Tagging

Rule-Based Tagging

Rule-based tagging uses manually defined linguistic rules.

Example:

“If ‘book’ occurs after a determiner, tag it as NN.”

Advantages:

- Easy to understand and interpret.
- Linguistic knowledge can be directly incorporated.
- Effective for well-defined grammatical patterns.
- Does not necessarily require a large annotated corpus.

Limitations:

- Creating and maintaining many rules is difficult.
- Rules may fail for previously unseen sentence structures.
- Large-scale systems require a very large number of rules.

Stochastic HMM Tagging

HMM tagging uses probabilities learned from training data.

It considers:

Transition Probability: Probability of one POS tag following another.

Emission Probability: Probability of a word occurring with a particular POS tag.

Advantages:

- Learns patterns from large datasets.
- Handles ambiguous words probabilistically.
- Can process large amounts of text efficiently.
- Provides systematic decisions for unseen contexts.

Limitations:

- Requires annotated training data.
- Performance depends on the quality and quantity of training data.
- Basic HMM assumptions may not capture all long-range linguistic dependencies.

Recommended Approach

For a large-scale customer support chatbot, stochastic tagging is generally more suitable than a purely rule-based system because it can learn from large datasets and handle lexical ambiguity probabilistically. A hybrid system can be even more effective, where statistical models make the primary predictions while carefully designed linguistic rules handle specific exceptional cases.

4. Role of Standardized POS Tagsets and Word Classes

Standardized POS tagsets such as the Penn Treebank tagset provide consistent grammatical labels for words.

Examples include:

NN – Noun

VB – Verb

JJ – Adjective

RB – Adverb

DT – Determiner

VBZ – Verb, third-person singular present

Importance in Intent Detection

POS tags help identify important actions and entities.

For example:

“Book a flight.”

The VB tag identifies book as an action, while NN tags can help identify entities such as flight and ticket.

This helps the chatbot identify that the user intends to make a booking.

Importance in Response Generation

Grammatical information helps the chatbot generate responses with appropriate sentence structure and word forms.

Handling Ambiguity

Words such as “book” can function as multiple word classes. POS tagging helps the chatbot distinguish between:

book/NN → an object

and

book/VB → an action

Overall Performance

Standardized tagsets make linguistic analysis consistent across large datasets. They support syntactic parsing, information extraction, intent classification, and response generation. Consequently, accurate POS tagging improves the overall reliability and naturalness of a customer support chatbot.

Conclusion

POS tagging is an important component of intelligent chatbot systems. Penn Treebank tags provide standardized grammatical categories, while HMM tagging uses probabilities to resolve lexical ambiguity. Context and probabilistic information correctly identify “book” as a verb in an imperative booking request and as a noun when referring to an object.

CASE STUDY 3: NEWS ANALYTICS AND POS TAG CORRECTION SYSTEM

Introduction

Transformation-Based Tagging, commonly associated with the Brill Tagger, improves an initial POS-tagged sequence by applying learned transformation rules. Instead of accepting the initial tagger's predictions directly, the system identifies systematic errors and corrects them according to contextual patterns.

Given Sentence:

“Economic growth increases employment.”

Initial POS Tags:

economic/JJ

growth/NN

increases/NNS

employment/NN

Transformation Rule:

Change NNS to VBZ if the preceding word is tagged as NN.

1. Application of Transformation Rule

Initial sequence:

economic/JJ growth/NN increases/NNS employment/NN

The word “increases” is initially tagged as:

increases/NNS

The preceding word is:

growth/NN

According to the learned rule:

NNS → VBZ if the preceding word is NN

Therefore:

increases/NNS → increases/VBZ

Corrected POS Sequence

economic/JJ growth/NN increases/VBZ employment/NN

Linguistic Explanation

In the sentence:

“Economic growth increases employment.”

“growth” is the subject and “increases” expresses the action performed by the subject.

Therefore, “increases” should be a verb rather than a plural noun.

Since “growth” is a third-person singular noun, the present-tense verb “increases” receives the Penn Treebank tag:

VBZ – Verb, third-person singular present

Thus, the transformation is linguistically appropriate.

2. Analysis of Initial Tagging Error

The initial tagger produced:

economic/JJ growth/NN increases/NNS employment/NN

The main error is:

increases/NNS

The tag NNS represents a plural noun.

The word ending “-s” may have caused the initial system to interpret “increases” as a plural noun.

However, its grammatical role in this sentence is a verb.

Consider the sentence structure:

Economic growth | increases | employment

Subject → Economic growth

Verb → increases

Object → employment

Therefore, the corrected sequence is:

economic/JJ growth/NN increases/VBZ employment/NN

The assignments are justified as follows:

economic/JJ: “economic” describes the noun “growth”, so it functions as an adjective.

growth/NN: “growth” is a singular noun and functions as the subject.

increases/VBZ: “increases” is a third-person singular present-tense verb.

employment/NN: “employment” is a noun and functions as the object of the verb.

This demonstrates why grammatical context is important when resolving POS-tagging ambiguity.

3. Word Frequency Distribution

Given corpus frequencies:

economic = 120

growth = 450

increases = 210

employment = 380

Step 1: Calculate Total Frequency

Total = 120 + 450 + 210 + 380

Total = 1160

Step 2: Calculate Relative Frequency

The formula is:

$P(\text{word}) = \text{Frequency of word} / \text{Total frequency}$

Economic

$P(\text{economic}) = 120 / 1160$

$= 0.1034 = 10.34\%$

Growth

$P(\text{growth}) = 450 / 1160$

$= 0.3879 = 38.79\%$

Increases

$P(\text{increases}) = 210 / 1160$

$= 0.1810 = 18.10\%$

Employment

$P(\text{employment}) = 380 / 1160$

$= 0.3276 = 32.76\%$

Frequency Distribution Table

Word	Frequency	Relative Frequency	Percentage
economic	120	0.1034	10.34%
growth	450	0.3879	38.79%
increases	210	0.1810	18.10%
employment	380	0.3276	32.76%
Total	1160	1.0000	100%

Role of Frequency in POS Tagging

Frequency information helps a probabilistic POS tagger estimate how commonly words and word-tag combinations occur in a corpus.

A frequently observed word-tag combination can receive a higher probability than a rare combination. However, word frequency alone cannot always determine the correct POS tag because many words are ambiguous.

Therefore, effective probabilistic POS tagging combines:

Word frequency + Tag frequency + Context + Transition probabilities

For example, even if “increases” appears frequently as a noun in some contexts, its position after the singular noun “growth” and before the noun “employment” provides strong grammatical evidence that it functions as a verb in the given sentence.

4. Transformation-Based Tagging and Entropy

Transformation-Based Tagging

Transformation-Based Tagging begins with an initial POS-tagged sequence and then applies learned correction rules.

In this example:

Initial:

economic/JJ growth/NN increases/NNS employment/NN

Rule:

Change NNS to VBZ if the preceding word is NN.

Corrected:

economic/JJ growth/NN increases/VBZ employment/NN

The transformation corrects the grammatical role of “increases” using contextual information.

Advantages

Transformation-Based Tagging:

- Corrects systematic mistakes produced by the initial tagger.
- Uses contextual information from neighboring words and tags.
- Produces interpretable correction rules.
- Combines an initial statistical or lexical tagging stage with rule-based refinement.
- Can improve overall POS-tagging accuracy when useful transformation rules have been learned from training data.

Entropy and Tagging Uncertainty

Entropy measures uncertainty in a probability distribution.

The general formula is:

$$H(X) = -\sum P(x) \log_2 P(x)$$

Suppose an initial tagger considers two possible tags for “increases”:

NNS and VBZ.

If their probabilities are similar, the entropy will be relatively high because the model is uncertain.

For example:

$$P(\text{NNS}) = 0.50$$

$$P(\text{VBZ}) = 0.50$$

Then:

$$H = -[0.5 \log_2(0.5) + 0.5 \log_2(0.5)]$$

$$H = 1 \text{ bit}$$

This represents maximum uncertainty between two equally likely tags.

After applying a strong contextual transformation rule, the system may become more confident in VBZ.

For illustration:

$$P(\text{VBZ}) = 0.95$$

$$P(\text{NNS}) = 0.05$$

Then:

$$H = -[0.95 \log_2(0.95) + 0.05 \log_2(0.05)]$$

$$H \approx \mathbf{0.286 \text{ bits}}$$

Thus:

Before correction → Higher entropy → Higher uncertainty

After correction → Lower entropy → Higher confidence

The exact before-and-after entropy cannot be calculated from the case-study data because tag probabilities are not supplied. The values above are illustrative examples showing how entropy can be used.

Evaluation

Transformation-Based Tagging improves the initial tagging stage by identifying systematic errors and applying context-sensitive corrections. In the given sentence, the initial NNS assignment is corrected to VBZ based on the preceding NN tag.

Entropy can complement this process by measuring the uncertainty associated with competing POS tags. A reduction in entropy after a reliable correction indicates that the system has become more confident in its final tag assignment.

Conclusion

Transformation-Based Tagging provides an effective mechanism for correcting errors generated during initial POS tagging. In the given example, the Brill-style transformation correctly changes “increases/NNS” to “increases/VBZ”. Corpus frequency information supports probabilistic decisions, while entropy provides a quantitative measure of uncertainty. Together, contextual rules, frequency analysis, and uncertainty measurement can improve the accuracy of large-scale news analytics systems.

OVERALL CONCLUSION

The three case studies demonstrate important Natural Language Processing techniques used in real-world AI applications. N-gram language models, backoff, interpolation, and entropy support intelligent word prediction in mobile keyboards. Penn Treebank POS tags and Hidden Markov Models help chatbots identify grammatical roles and resolve lexical ambiguity. Transformation-Based Tagging improves initial POS-tagging results by applying learned contextual corrections. These techniques enable NLP systems to handle uncertainty, sparse data, ambiguous words, and complex grammatical structures more effectively.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____